

# VIDEO TRANSCRIPT — The Ledger: Agentic Event Store & Enterprise Audit Infrastructure

---

Max 6 minutes | Week 5 TRP1 FDE Program

---

## SETUP (Before Recording — Do This First)

---

Open two terminal panes side by side. In both, **cd** to the repo root:

```
cd "/Users/natnaelalemseged/code-projects/education/10 academy/Axiom Ledger"
```

Have your editor open showing **src/event\_store.py** and **src/mcp/tools.py** in tabs — you'll reference them visually.

Clear both terminals. Zoom font to ~18pt so output is readable on screen.

---

## MINUTE 0:00–0:20 — COLD OPEN / FRAME THE PROBLEM

---

*[Camera on face, then screen share]*

"Regulators, auditors, and enterprise risk teams need one thing before they'll approve an AI system in production: an immutable record of every AI decision, and the data that informed it. Without that, the system doesn't go live. I built The Ledger — an append-only, event-sourced infrastructure that makes every agent action cryptographically traceable, temporally queryable, and architecturally auditable. Let me show you what that means in practice."

*[Cut to terminal. Show the test suite summary briefly.]*

```
.venv/bin/python -m pytest tests/ -q --no-header 2>&1 | tail -3
```

*Expected output:*

```
55 passed, 8 skipped in 0.82s
```

"55 tests passing across all six phases — event store, aggregates, projections, upcasting, Gas Town recovery, and the MCP interface. The 8 skipped require a live Postgres connection, which I'll demonstrate separately. Let's get into the demo."

---

## MINUTE 0:20–1:30 — STEP 1: THE WEEK STANDARD

---

"The Week Standard for this challenge is: 'Show me the complete decision history of application ID X — from submission to final decision, every AI agent action, every compliance check, causal links intact, and cryptographic integrity verified.' It must run in under 60 seconds. I'm timing it now."

*[Start a visible timer or call it out. Type:]*

```
.venv/bin/python -m pytest tests/test_mcp_lifecycle.py::test_full_mcp_loan_lifecycle -v --no-header 2>&1 | grep -E "P
```

"Let me walk you through what that test actually does — this is the full lifecycle driven entirely through MCP tool calls. No direct Python calls. Exactly what a real AI agent would do."

[Open **tests/test\_mcp\_lifecycle.py** in editor side-by-side. Scroll through it slowly while narrating:]

"Step 1: The Gas Town anchor. Before any agent does any work, it appends **start\_agent\_session**. This is the crash-recovery anchor — if the process dies, the session stream is its memory."

"Step 2: **submit\_application** — the **ApplicationSubmitted** event hits the **loan-{id}** stream. The event store enforces optimistic concurrency from the first write."

"Step 3: **record\_credit\_analysis**. The MCP tool validates: does this agent have an active session? If not, it returns a structured **PreconditionFailed** error — not a string, a typed object the LLM can reason about."

"Step 4: Fraud screening. Step 5: Compliance check against FINCEN-2026 regulations. Step 6: Decision generated — confidence 0.82, recommendation APPROVE. Step 7: Human officer confirms. The application is **APPROVED**."

"Then we query two resources: the compliance audit view and the full audit trail. Every event type is present — **ApplicationSubmitted**, **CreditAnalysisCompleted**, **HumanReviewCompleted**, **ApplicationApproved**. The trace is complete."

[Run just the lifecycle test, watching it pass:]

```
.venv/bin/python -m pytest tests/test_mcp_lifecycle.py::test_full_mcp_loan_lifecycle -v --no-header -s 2>&1 | grep -v
```

Expected output:

```
tests/test_mcp_lifecycle.py::test_full_mcp_loan_lifecycle PASSED
```

"Complete loan lifecycle through MCP only. That's Step 1 done. Now — the confidence floor enforcement is a regulatory requirement baked into the aggregate, not the API layer."

[Run confidence floor test:]

```
.venv/bin/python -m pytest tests/test_mcp_lifecycle.py::test_confidence_floor_forces_refer -v --no-header 2>&1 | grep
```

"Caller sends confidence 0.45 and tries to APPROVE. The aggregate overrides it to REFER. The LLM cannot bypass this — it's enforced in domain logic."

---

## MINUTE 1:30–2:30 — STEP 2: CONCURRENCY UNDER PRESSURE

---

"Here's the production scenario that breaks most event sourcing implementations. Two AI fraud-detection agents simultaneously process the same loan application. Both read the stream at version 3. Both call append with **expected\_version=3**. Without optimistic concurrency control, you get split-brain — two fraud scores, no authoritative state. Let me show you what The Ledger does."

[Type:]

```
.venv/bin/python -m pytest tests/test_concurrency.py::test_double_decision_exactly_one_wins -v --no-header -s 2>&1 |
```

Expected output:

```
tests/test_concurrency.py::test_double_decision_exactly_one_wins PASSED
```

"Exactly one agent wins. The other receives a typed **OptimisticConcurrencyError** — with **stream\_id**, **expected\_version**, **actual\_version**, and **suggested\_action: reload\_stream\_and\_retry**. The total stream length is 5 — not 6. One event was appended. The losing agent must reload and re-evaluate."

[Open **tests/test\_concurrency.py** in the editor, scroll to the assertion block:]

"The assertions: stream length equals before-length plus one. Success positions equal exactly [4]. One **OptimisticConcurrencyError** raised, not silently swallowed. This is not an edge case — at 1,000 applications per hour with four agents each, collisions happen constantly. The Ledger handles it without locks, without transactions spanning aggregates."

---

## MINUTE 2:30–3:00 — STEP 3: TEMPORAL COMPLIANCE QUERY

---

"Regulators don't just want current state. They want: 'What did this application look like on March 15th?' The **ComplianceAuditView** projection supports time-travel queries."

[Run the temporal query test:]

```
.venv/bin/python -m pytest tests/test_projections.py::test_compliance_audit_temporal_query -v --no-header -s 2>&1 | g
```

Expected output:

```
tests/test_projections.py::test_compliance_audit_temporal_query PASSED
```

"This test writes compliance events across a time range, then queries **get\_compliance\_at(application\_id, timestamp)** for a past moment. The result shows compliance state as it existed then — not now. A rule that passed later doesn't appear in the earlier snapshot. That's the temporal query guarantee."

[Point to **src/projections/compliance\_audit.py** in editor:]

"The resource is **ledger://applications/{id}/compliance?as\_of=2026-03-15T09:00:00Z**. It reads from a projection table, never replaying aggregate streams — p99 under 200 milliseconds by SLO."

"That's the three required steps in under three minutes. Now — the mastery demonstrations."

---

## MINUTE 3:00–4:00 — STEP 4: UPCASTING IMMUTABILITY

---

"Event sourcing's core guarantee: the past is immutable. But schemas evolve. In 2024, **CreditAnalysisCompleted** had three fields. In 2026 it needs six. A CRUD system would run a migration and mutate the data. An event store cannot. Instead, we use upcasters — functions that transform old events at read time, never touching the stored bytes."

[Run the upcasting immutability test:]

```
.venv/bin/python -m pytest tests/test_upcasting.py::test_credit_analysis_upcasting_immutability -v --no-header -s 2>&1 | g
```

Expected output:

```
tests/test_upcasting.py::test_credit_analysis_upcasting_immutability PASSED
```

"Three assertions. One: we store a v1 event directly. Two: we load it through **EventStore.load\_stream()** — it comes back as v2, with **regulatory\_basis** added, **model\_version** inferred from the **recorded\_at** timestamp. Three: we reach into **\_streams** and read the raw stored dict — it is still version 1. **regulatory\_basis** is not there. The stored bytes are unchanged."

[Open **src/upcasting/upcasters.py** briefly:]

"For fields we genuinely cannot infer — like **confidence\_score** on historical events from before we tracked it — we return **null**, not a fabricated value. The DESIGN.md documents exactly why: an incorrect inference has a quantifiable downstream consequence. Null is honest; fabrication is a compliance liability."

[Run all upcasting tests:]

```
.venv/bin/python -m pytest tests/test_upcasting.py -v --no-header 2>&1 | grep -E "PASSED|FAILED"
```

Expected output:

```
tests/test_upcasting.py::test_credit_analysis_upcasting_immutability PASSED
tests/test_upcasting.py::test_decision_generated_upcasting_immutability PASSED
tests/test_upcasting.py::test_upcaster_registry_returns_new_dict PASSED
tests/test_upcasting.py::test_upcaster_chain_multiple_versions PASSED
tests/test_upcasting.py::test_upcaster_no_op_for_current_version PASSED
```

"Five upcasting tests passing — including a v1→v2→v3 chain and a no-op guard for events already at current version."

---

## MINUTE 4:00–5:00 — STEP 5: GAS TOWN CRASH RECOVERY

---

"Named for the infrastructure anti-pattern where agent context is lost on process restart. The Gas Town pattern is the fix: every agent action is written to the event store before it executes. On crash, a new agent instance replays its session stream to reconstruct context and resume — without redoing completed work. Let me show the recovery test."

[Run the Gas Town tests:]

```
.venv/bin/python -m pytest tests/test_gas_town.py -v --no-header -s 2>&1 | grep -v "DeprecationWarning\|asyncio\|plug"
```

Expected output:

```
tests/test_gas_town.py::test_agent_context_reconstruction_after_crash PASSED
tests/test_gas_town.py::test_needs_reconciliation_for_incomplete_session PASSED
tests/test_gas_town.py::test_empty_session_returns_empty_status PASSED
tests/test_gas_town.py::test_context_respects_token_budget PASSED
```

"The crash test: we append 5 agent session events — **AgentContextLoaded**, then three **AgentNodeExecuted** events for **load\_documents**, **compute\_ratios**, and **llm\_analysis**, plus an **AgentToolCalled**. Then we drop the in-memory agent object entirely — simulating a process kill. We call **reconstruct\_agent\_context()** cold."

[Point to **src/integrity/gas\_town.py** in editor:]

"The reconstructed context has: the application ID, the model version, the number of events replayed, the last event position, the session health status — **HEALTHY** because the session completed normally. And a **context\_text** string summarizing the session history within the token budget — ready to inject directly into the resumed agent's prompt."

"Second test: the session ended with **AgentSessionFailed** and no recovery event. Status is **NEEDS\_RECONCILIATION**. The agent cannot resume blindly — it must resolve the partial state first. This is enforced by the context health check, not left to the agent's discretion."

"Third test: a token budget of 500 tokens. The context text is truncated — but the most recent 3 events and any **PENDING** or **ERROR** state events are always preserved verbatim. Older events are summarized to prose."

---

## MINUTE 5:00–6:00 — STEP 6 (BONUS): WHAT-IF COUNTERFACTUAL + CLOSE

---

"The bonus phase: regulatory counterfactual analysis. The compliance team asks — 'What would the final decision have been if the credit agent had returned risk tier HIGH instead of MEDIUM?' This is not a hypothetical exercise. Regulators require it for model audits."

*[Run narratives which exercise the full pipeline:]*

```
.venv/bin/python -m pytest tests/test_narratives.py -v --no-header 2>&1 | grep -E "PASSED|FAILED"
```

*Expected output:*

```
tests/test_narratives.py::test_narr01 PASSED
tests/test_narratives.py::test_narr02 PASSED
tests/test_narratives.py::test_narr03 PASSED
tests/test_narratives.py::test_narr04 PASSED
tests/test_narratives.py::test_narr05 PASSED
tests/test_narratives.py::test_deliverable_five_terminal_states PASSED
```

*[Open **src/what\_if/projector.py** in editor, scroll to the **run\_what\_if** function:]*

"The what-if projector loads all real events up to the branch point — **CreditAnalysisCompleted**. It injects a counterfactual event with **risk\_tier=HIGH** instead of **MEDIUM**. It then continues replaying real events that are causally independent of the branch — events whose **causation\_id** does not trace back to the substituted event. Events that are causally dependent — like **DecisionGenerated** which referenced the original credit session — are skipped."

"The result: real outcome APPROVE versus counterfactual outcome REFER. The confidence floor rule cascades. No counterfactual events are written to the real store. The live database is untouched."

*[Show **src/regulatory/package.py** briefly:]*

"And finally — the regulatory examination package. A single JSON file: the complete event stream in order, projection states at the examination date, audit chain integrity verification, a human-readable narrative generated by replaying events, and every AI agent's model version and input data hash. A regulator can verify this package independently without trusting our system."

*[Run the full test suite one final time, zoomed in on the summary line:]*

```
.venv/bin/python -m pytest tests/ -q --no-header 2>&1 | tail -3
```

*Expected output:*

```
55 passed, 8 skipped in 0.82s
```

"55 tests. Six phases. Event store with optimistic concurrency. CQRS projections with sub-500ms lag SLOs. Upcasting that never touches stored events. Crash recovery from event replay. MCP tools and resources for every command and query. Cryptographic audit chains. Counterfactual what-if analysis. Regulatory examination packages."

"Auditability is not an annotation added after the fact. It is the architecture. That's The Ledger."

[End recording]

## QUICK REFERENCE: Command Cheat Sheet

| Demo Step              | Command                                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------|
| Full test suite        | <code>`\` .venv/bin/python -m pytest tests/ -q --no-header 2&gt;&amp;1 \   tail -3`</code>                             |
| MCP full lifecycle     | <code>.venv/bin/python -m pytest tests/test_mcp_lifecycle.py::test_full_mcp_loan_lifecycle -v --no-header</code>       |
| Confidence floor       | <code>.venv/bin/python -m pytest tests/test_mcp_lifecycle.py::test_confidence_floor_forces_refer -v --no-header</code> |
| Concurrency            | <code>.venv/bin/python -m pytest tests/test_concurrency.py -v --no-header</code>                                       |
| Temporal query         | <code>.venv/bin/python -m pytest tests/test_projections.py::test_compliance_audit_temporal_query -v --no-header</code> |
| Upcasting immutability | <code>.venv/bin/python -m pytest tests/test_upcasting.py -v --no-header</code>                                         |
| Gas Town recovery      | <code>.venv/bin/python -m pytest tests/test_gas_town.py -v --no-header</code>                                          |
| Narrative + what-if    | <code>.venv/bin/python -m pytest tests/test_narratives.py -v --no-header</code>                                        |

## GRADING RUBRIC COVERAGE

| Criterion                         | Where Demonstrated                                                                      |
|-----------------------------------|-----------------------------------------------------------------------------------------|
| Event store schema + OCC          | Step 2 concurrency test — one winner, typed error                                       |
| Aggregate replay + state machine  | Step 1 MCP lifecycle — every transition enforced                                        |
| Business rules in domain, not API | Confidence floor (0.45 → REFER) — domain enforces                                       |
| Gas Town precondition             | <code>test_record_credit_without_session_fails</code> — <code>PreconditionFailed</code> |
| Projections + lag SLO             | Step 3 temporal query — <code>test_projection_daemon_lag_under_load</code>              |
| Upcasting immutability            | Step 4 — raw stored v1 unchanged after loading as v2                                    |
| Crash recovery                    | Step 5 — <code>reconstruct_agent_context()</code> cold, HEALTHY status                  |
| MCP CQRS design                   | Step 1 — tools=Commands, resources=Queries, never replay on read                        |
| DESIGN.md / DOMAIN_NOTES          | Referenced verbally — point to <code>docs/DESIGN.md</code> on screen                    |
| Bonus: What-if counterfactual     | Step 6 — <code>run_what_if()</code> , causal filtering, no writes to real store         |
| Bonus: Regulatory package         | Step 6 — <code>generate_regulatory_package()</code> , self-contained JSON               |

## KEY PHRASES TO MEMORIZE

---

Say these verbatim — they map directly to rubric language:

- *"Auditability is the architecture, not an annotation."*
- *"The aggregate enforces the confidence floor — the LLM cannot override it."*
- *"One append succeeds. The other receives a typed **OptimisticConcurrencyError** with **suggested\_action: reload\_stream\_and\_retry**."*
- *"Upcasting transforms events at read time. The stored bytes are never touched."*
- *"The session stream is the agent's memory. On crash, replay the stream — no work is repeated."*
- *"Counterfactual events are never written to the real store. The live database is untouched."*